# CHIANGMAI AIR QUALITY ALERT SYSTEM: END-TO-END MLOPS PIPELINE FOR PM2.5 FORECASTING

*Progress Report*

by

Supanut Kompayak (st126055)

Shuvam Shrestha (st125975)

A Progress Report Submitted in Partial Fulfillment of the Requirements for the Course AT82.9002 – Selected Topic: Data Engineering and MLOps

Instructor: Dr. Chantri Polprasert

Course: AT82.9002 Selected Topic: Data Engineering and MLOps

Semester: January 2026

Asian Institute of Technology
School of Engineering and Technology
Thailand

March 2026

# ABSTRACT

Chiang Mai, Thailand, consistently ranks among the most severely air-polluted cities in the world during its annual haze season from January to April. Fine particulate matter (PM2.5) regularly exceeds the World Health Organization's 24-hour guideline of 15 µg/m$^3$ by a factor of three or more. Despite the predictability of these pollution episodes, existing public-facing tools provide only real-time monitoring with no forward-looking predictive capability.

This progress report presents the current implementation status of the ChiangMai Air Quality Alert System (CAAS), a production-ready end-to-end MLOps pipeline that forecasts PM2.5 concentration levels 1, 3, and 7 days in advance and automatically triggers health alerts when projected concentrations are expected to exceed 50 µg/m$^3$. The system integrates three complementary data sources: historical PM2.5 observations from Thailand's Pollution Control Department (PCD), meteorological variables from the Open-Meteo API, and fire hotspot detections from NASA FIRMS. A 15-year consolidated dataset spanning 2011 to 2025 (5,478 daily observations, 4.0% missing) has been assembled and validated. Exploratory analysis confirms strong seasonal structure: March mean PM2.5 of 78.0 µg/m$^3$, 731 hazardous days (>50 µg/m$^3$) concentrated in January–April, and high short-term autocorrelation ($r = 0.925$ at lag-1). A persistence baseline establishes the performance floor: MAE of 5.30 (t+1), 8.97 (t+3), and 10.93 µg/m$^3$ (t+7).

This report also addresses all feedback received during the proposal presentation, including unifying the drift and retraining trigger logic, resolving cost inconsistencies, describing the GitHub Actions CI/CD pipeline in detail, and reframing the project's contribution accurately as a production-grade MLOps integration rather than a novel forecasting methodology. Feature engineering (45 features), XGBoost training, and LSTM training scripts have been completed. Full model training, evaluation, and cloud deployment are in progress.

# Contents

**ABSTRACT** **i**

**LIST OF ABBREVIATIONS** **vii**

**1 INTRODUCTION** **1**

1.1 Background of the Study . . . . . 1
1.2 Statement of the Problem . . . . . 1
1.3 Objectives of the Study . . . . . 2
1.4 Organization of the Report . . . . . 3

**2 BACKGROUND AND RELATED WORK** **4**

2.1 PM2.5 Air Pollution and Public Health . . . . . 4
2.2 Data-Driven Approaches for Air Quality Forecasting . . . . . 4
2.3 Tabular Models versus Sequential Models for Time Series . . . . . 5
2.4 MLOps: Principles and Tools . . . . . 5
2.5 Related Systems and Novelty . . . . . 6

**3 PROPOSED METHODOLOGY** **7**

3.1 System Overview . . . . . 7
3.2 Data Pipeline Design . . . . . 7
3.2.1 Data Sources . . . . . 7
3.2.2 Ingestion Method . . . . . 8
3.2.3 Transformation Pipeline . . . . . 8
3.3 Feature Engineering . . . . . 8
3.3.1 Temporal Features . . . . . 9
3.3.2 Contextual Environmental Features . . . . . 10
3.3.3 Target Variables . . . . . 10
3.4 Model Training and Evaluation . . . . . 10
3.4.1 Model Selection Rationale . . . . . 10
3.4.2 Chronological Data Split . . . . . 12

**4 SYSTEM ARCHITECTURE AND PIPELINE DESIGN** **13**

4.1 Architecture Overview . . . . . 13

4.2 Storage Layers . . . . . 13
4.3 MLOps Lifecycle . . . . . 14
4.3.1 Experiment Tracking . . . . . 14
4.3.2 Model Versioning and Promotion . . . . . 14
4.3.3 GitHub Actions CI/CD Retraining Pipeline . . . . . 15
4.3.4 Deployment Strategy . . . . . 16
4.4 Monitoring and Automated Retraining . . . . . 18
4.4.1 Drift Detection Signals . . . . . 18
4.4.2 Retraining Validation Gate . . . . . 18
4.4.3 Evidently AI Monitoring Configuration . . . . . 19
4.5 Infrastructure and Cost Estimate . . . . . 19
4.6 Data Quality Management . . . . . 20
4.6.1 Missing Value Handling Strategy . . . . . 21
4.6.2 Schema Validation Rules . . . . . 21
4.6.3 Pipeline Health Monitoring . . . . . 21

**5 EXPERIMENTAL PLAN AND EXPECTED CONTRIBUTIONS 22**
5.1 Dataset Characteristics . . . . . 22
5.2 Baseline Methods for Comparison . . . . . 22
5.3 Evaluation Metrics . . . . . 23
5.4 Experimental Setup . . . . . 23
5.5 Expected Contributions . . . . . 23
5.6 Feasibility and Tool Selection . . . . . 24
5.7 Team Roles and Responsibilities . . . . . 25
5.8 Known Limitations and Risk Mitigation . . . . . 25
5.9 Revised Project Timeline . . . . . 27

**6 IMPLEMENTATION PROGRESS 28**
6.1 Data Collection Status . . . . . 28
6.1.1 PM2.5 Data: Station Merging Logic . . . . . 28
6.1.2 NASA FIRMS Integration . . . . . 29
6.2 Scripts Inventory . . . . . 29
6.3 Project Folder Structure . . . . . 29

**7 PRELIMINARY RESULTS 31**
7.1 PM2.5 Dataset Characteristics . . . . . 31
7.1.1 Overall Statistics . . . . . 31
7.1.2 Seasonal Pattern . . . . . 32
7.1.3 Annual Trends . . . . . 33
7.2 Temporal Autocorrelation Analysis . . . . . 33

7.3 Persistence Baseline Evaluation . . . . . 34
7.4 Target Performance Benchmarks . . . . . 35

**8 DETAILED EXPERIMENTAL SCENARIOS 36**
8.1 Overview of Experimental Design . . . . . 36
8.2 Scenario A: XGBoost versus Persistence Baseline . . . . . 36
8.3 Scenario B: XGBoost versus LSTM by Horizon . . . . . 37
8.4 Scenario C: Alert Classification . . . . . 38
8.5 Scenario D: Feature Ablation — FIRMS Fire Hotspot Signal . . . . . 38
8.6 MLflow Experiment Organisation . . . . . 39

**REFERENCES 39**

# List of Tables


1.1 Problem framing summary . . . . . 2

3.1 Summary of integrated data sources . . . . . 7
3.2 Temporal autocorrelation and calendar features . . . . . 9
3.3 Contextual meteorological and fire activity features . . . . . 10
3.4 Target variables for regression and classification tasks . . . . . 10
3.5 Chronological training, validation, and test split . . . . . 12

4.1 GitHub Actions retraining pipeline stages . . . . . 16
4.2 Unified drift detection signals, thresholds, and actions . . . . . 18
4.3 Revised monthly AWS infrastructure cost breakdown . . . . . 20
4.4 Missing value handling by data source . . . . . 21

5.1 Evaluation metrics by task type and priority . . . . . 23
5.2 Team member roles and task ownership . . . . . 25
5.3 Known limitations and risk mitigation strategies . . . . . 26
5.4 Updated project milestones with completion status . . . . . 27

6.1 Data collection and processing status as of March 27, 2026 . . . . . 28
6.2 Completed scripts in `04_Scripts/` . . . . . 29

7.1 Descriptive statistics for PM2.5 dataset (2011–2025, $n = 5{,}257$ valid days) . . . . . 31
7.2 Average daily PM2.5 by month (2011–2025 mean) . . . . . 32
7.3 Annual PM2.5 statistics (2011–2025) . . . . . 33
7.4 PM2.5 autocorrelation at selected lags . . . . . 34
7.5 Persistence baseline performance across full dataset (2011–2025) . . . . 34
7.6 Persistence baseline performance during haze season (January–April only) 34
7.7 Alert classification: persistence baseline for hazard alert (>50 μg/m$^3$ threshold) . . . . . 35
7.8 Minimum performance targets for XGBoost and LSTM models . . . . . 35

8.1 Summary of experimental scenarios . . . . . 36
8.2 Alert classification evaluation targets per horizon . . . . . 38

# List of Figures

3.1 High-level overview of the ChiangMai Air Quality Alert System (CAAS). 7

4.1 AWS cloud architecture for the ChiangMai Air Quality Alert System (CAAS). The pipeline spans three external data sources (air4thai API, Open-Meteo API, NASA FIRMS API), an AWS Lambda ingestion layer triggered daily at 06:00 UTC+7 via GitHub Actions, four S3 storage tiers (Raw, Processed, Feature Store, Model Artifacts), and a compute layer comprising MLflow Tracking Server, FastAPI Inference Server, Streamlit Dashboard, and Evidently AI drift monitoring. Dashed arrows indicate orchestration triggers and automated retraining signals. . . 13

4.2 MLflow model promotion and lifecycle flow. Newly trained models are registered in MLflow and enter the Staging stage, where they undergo schema validation (44-feature check), performance validation ($R^2 \geq 0.70$), and regression tests against the current Production model. Failed validation keeps the model in Staging with an alert logged. On success, the model is promoted to Production with zero-downtime FastAPI hot-reload; the previous Production model is archived for rollback. Evidently AI drift detection and a daily scheduled pipeline serve as the primary retraining triggers. . . . . 15

4.3 Daily MLOps pipeline execution flow triggered at 06:00 UTC+7 via GitHub Actions. The pipeline comprises seven stages: (1) Data Ingestion via AWS Lambda fetching from air4thai, Open-Meteo, and NASA FIRMS APIs into S3 Raw; (2) Schema Validation with a failure branch that sends an alert and stops the pipeline; (3) Data Cleaning including duplicate removal, forward-fill for missing values, and outlier clipping; (4) Feature Engineering producing 44 features (lag, rolling statistics, cyclical encodings, seasonal flags, fire hotspots) stored to S3 Feature Store; (5) Model Inference generating PM2.5 regression values and alert-level classifications for t+1, t+3, and t+7 horizons; (6) Store Predictions to S3 and update the FastAPI cache; (7) Monitoring Check using Evidently AI PSI drift detection, flagging for retraining if PSI $> 0.2$. 17

4.4 End-to-end PM2.5 forecast request flow for the CAAS system. . . . . 17

# LIST OF ABBREVIATIONS

| | |
|---|---|
| API | Application Programming Interface |
| AQI | Air Quality Index |
| AWS | Amazon Web Services |
| CAAS | ChiangMai Air Quality Alert System |
| CI/CD | Continuous Integration / Continuous Deployment |
| EC2 | Elastic Compute Cloud (AWS) |
| FIRMS | Fire Information for Resource Management System |
| FRP | Fire Radiative Power |
| LSTM | Long Short-Term Memory |
| MAE | Mean Absolute Error |
| ML | Machine Learning |
| MLOps | Machine Learning Operations |
| PCD | Pollution Control Department (Thailand) |
| PM2.5 | Particulate Matter with diameter $\leq$2.5 µm |
| PSI | Population Stability Index |
| REST | Representational State Transfer |
| RMSE | Root Mean Squared Error |
| S3 | Amazon Simple Storage Service |
| SHAP | SHapley Additive exPlanations |
| WHO | World Health Organization |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background of the Study

Air quality degradation from fine particulate matter (PM2.5) represents one of the most pressing environmental health challenges in Southeast Asia. PM2.5 particles, with aerodynamic diameters of 2.5 micrometers or less, penetrate deep into the respiratory system and are associated with increased incidence of respiratory disease, cardiovascular disease, and premature mortality [1]. The World Health Organization's revised 2021 air quality guidelines set the 24-hour mean PM2.5 threshold at 15 $\mu g/m^3$—a standard that Chiang Mai regularly exceeds by a factor of three to six during its haze season.

Chiang Mai's air quality crisis is driven by a convergence of factors: agricultural burning in the lowlands, wildfires in the surrounding highland forests, transboundary smoke from neighbouring countries (notably Myanmar and Laos), and the city's basin-shaped topography, which limits atmospheric dispersion and traps pollutants near the ground. These conditions create highly predictable annual pollution cycles, with peak concentrations typically occurring between January and April, punctuated by acute events triggered by concentrated fire activity.

Despite the predictability of these patterns, the public information infrastructure remains reactive rather than proactive. Thailand's PCD provides real-time AQI data through the air4thai platform, but no operational system exists to forecast upcoming pollution levels or automatically alert residents and health authorities before dangerous thresholds are breached. The resulting gap between available observational data and actionable early warning represents both a significant public health failure and a data engineering opportunity with real societal impact.

## 1.2 Statement of the Problem

The core problem addressed by this project is the absence of a predictive, automated early warning system for PM2.5 air quality events in Chiang Mai. Current tools inform the public about pollution that has already occurred; they cannot provide advance notice of conditions likely to develop over the coming one to seven days.

Specifically, the following capability gaps exist in the current landscape:

1. No operational system predicts PM2.5 concentration levels 1, 3, or 7 days in advance for Chiang Mai or Northern Thailand more broadly.

2. No automated mechanism exists to trigger health alerts before PM2.5 reaches hazardous thresholds, denying residents and public health agencies the lead time needed for protective action.
3. No self-updating system adapts to the shifting pollution patterns driven by changing agricultural practices, increasing wildfire frequency, and long-term climate trends.

Table 1.1: Problem framing summary

| **Dimension** | **Description** |
|---|---|
| Who is affected | Residents, tourists, public health agencies, city planners |
| Core problem | No predictive early warning system exists—only real-time monitoring |
| Impact | Preventable health exposure due to lack of advance notice |
| Gap | Existing tools show current conditions; none forecast what is coming |
| Proposed solution | End-to-end MLOps pipeline: daily forecast (t+1/3/7) + automated alert + drift-aware retraining |

### 1.3 Objectives of the Study

The primary objective of this project is to design and implement a production-ready MLOps pipeline providing accurate PM2.5 forecasts and automated health alerts for Chiang Mai. The specific objectives are:

1. To design and implement a multi-source data ingestion pipeline that automatically collects PM2.5, weather, and fire hotspot data on a daily basis.
2. To engineer a feature set capturing temporal autocorrelation, seasonal patterns, meteorological influences, and regional fire activity signals.
3. To train and empirically compare XGBoost and LSTM models for PM2.5 regression and alert classification across multiple forecast horizons (t+1, t+3, t+7).
4. To deploy the forecasting system as a REST API and Streamlit dashboard accessible to end users.
5. To implement a monitoring and automated retraining system detecting data drift and model performance degradation using Evidently AI and MLflow.

6. To demonstrate a cost-effective cloud deployment within AWS Learner Lab constraints ($2–8/month target).

### 1.4 Organization of the Report

Chapter 2 reviews relevant background literature on PM2.5 forecasting, time series modelling, and MLOps principles. Chapter 3 presents the revised methodology, addressing all feedback received during the proposal presentation. Chapter 4 details the system architecture and MLOps lifecycle, including the GitHub Actions retraining pipeline and unified drift detection logic. Chapter 5 describes the implementation progress achieved to date, including data collection, feature engineering, and training scripts. Chapter 6 presents preliminary results from exploratory data analysis and persistence baseline evaluation. Chapter 7 describes the detailed experimental scenarios and evaluation plan. Chapter 8 covers revised feasibility and cost analysis. Chapter 9 presents the updated project timeline and next steps.

# CHAPTER 2

# BACKGROUND AND RELATED WORK

## 2.1 PM2.5 Air Pollution and Public Health

Fine particulate matter (PM2.5) is among the most comprehensively studied environmental health hazards globally. Long-term epidemiological evidence demonstrates statistically robust associations between PM2.5 exposure and respiratory disease, ischemic heart disease, stroke, and lung cancer [1]. The WHO 2021 Global Air Quality Guidelines tightened the 24-hour PM2.5 guideline from 25 μg/m$^3$ to 15 μg/m$^3$, reflecting accumulated evidence of health impacts at lower concentrations than previously understood.

Northern Thailand's haze problem is characterised by sharp seasonal spikes driven primarily by biomass burning. Unlike urban vehicular pollution, which tends to be geographically diffuse and temporally consistent, fire-driven PM2.5 episodes are spatially concentrated and temporally clustered. This structure makes them, in principle, predictable from fire activity and meteorological precursors—a property that motivates the data-driven forecasting approach in this project.

## 2.2 Data-Driven Approaches for Air Quality Forecasting

Traditional approaches to air quality forecasting rely on physics-based chemical transport models (CTMs) such as WRF-Chem and CMAQ, which simulate atmospheric dynamics and chemistry explicitly [2]. While physically interpretable, these models are computationally intensive, require detailed emission inventories, and are difficult to adapt to local conditions without domain expertise.

Data-driven machine learning approaches have increasingly demonstrated competitive performance for short-range air quality forecasting, particularly when rich observational histories are available. Gradient boosting methods such as XGBoost and LightGBM perform strongly on tabular time series when temporal lag features are included, effectively encoding the autoregressive structure of pollution time series into a flat feature representation [4]. Studies in regional PM2.5 forecasting in China, India, and Southeast Asia report $R^2$ values in the range of 0.70–0.88 for next-day prediction using similar feature engineering approaches [3].

### 2.3 Tabular Models versus Sequential Models for Time Series

The relative merits of gradient boosting methods and deep sequential models (LSTM, Transformer) for tabular time series prediction have been an active area of benchmarking research. The M4 and M5 forecasting competitions demonstrated that tree-based and statistical ensemble methods consistently match or outperform deep learning approaches across a wide range of time series domains [6].

Shwartz-Ziv and Armon [7] conducted a systematic evaluation demonstrating that gradient boosting methods (XGBoost, CatBoost) generally outperform deep learning on tabular datasets with moderate sample sizes, particularly when domain-relevant features are explicitly engineered. Their findings are directly relevant to the dataset considered in this project: approximately 5,475 daily observations is a moderate-size tabular dataset by machine learning standards.

Hochreiter and Schmidhuber's Long Short-Term Memory architecture [5] was designed specifically to capture long-range temporal dependencies without the vanishing gradient problem of standard RNNs. LSTM has demonstrated advantages over tabular models in domains where the *sequence itself* contains information beyond what fixed-window lag features can encode—for example, in multi-step forecasting tasks where the future trajectory depends on complex interactions across long lookback periods.

**Implication for this project:** Given the dataset size and the fact that the feature engineering strategy already encodes autocorrelation explicitly through lag and rolling features, XGBoost is expected to perform well across all forecast horizons. LSTM is included as a comparative model to empirically test whether sequential modeling provides additional benefit—particularly at t+3 and t+7 horizons where fixed-lag features may capture less of the relevant dynamics. The production model selection will be determined by empirical evidence, not assumed in advance.

### 2.4 MLOps: Principles and Tools

MLOps is the practice of applying DevOps principles to machine learning systems to enable reliable, scalable, and maintainable deployment of ML models in production. Sculley et al. [8] identified the pervasive challenge of "hidden technical debt" in ML systems, noting that the model training code typically represents only a small fraction of total system complexity, with data pipelines, serving infrastructure, and monitoring comprising the majority of the codebase and operational burden.

Key operational challenges in production ML systems include: data distribution shift (data drift) causing model predictions to degrade over time, concept drift where the statistical relationship between inputs and targets changes, and the need for reproducible experiment tracking to enable model comparison and rollback [8].

This project addresses these challenges through a combination of MLflow for experiment tracking and model registry management [9], Evidently AI for automated drift detection, and GitHub Actions for CI/CD orchestration. The combination of these tools provides a lightweight but complete MLOps stack appropriate for the project scope and budget constraints.

### 2.5 Related Systems and Novelty

Thailand's PCD air4thai platform [10] provides real-time AQI data from approximately 90 monitoring stations nationwide but offers no forecasting capability. NASA FIRMS [11] provides near-real-time global fire detection data from MODIS and VIIRS satellite instruments, enabling quantification of fire activity as an explanatory variable for PM2.5 episodes. Open-Meteo [12] provides free retrospective and forecast meteorological data suitable for academic applications.

No published operational system to date combines these three data sources into an automated daily forecasting and alerting pipeline for Chiang Mai. The primary novelty of CAAS lies in integrating multi-source data acquisition, feature engineering, ML model training and comparison, automated deployment, and continuous monitoring within a single production-grade pipeline aligned with current MLOps best practices.

# CHAPTER 3

# PROPOSED METHODOLOGY

## 3.1 System Overview

The ChiangMai Air Quality Alert System (CAAS) is designed as a fully automated end-to-end MLOps pipeline. The system ingests data from three sources daily, generates PM2.5 regression forecasts and binary alert classifications for the next 1, 3, and 7 days, and triggers automated health alerts when concentrations are projected to exceed the 50 µg/m$^3$ threshold. A monitoring subsystem continuously evaluates forecast accuracy and detects data drift, triggering automated retraining when degradation is detected.

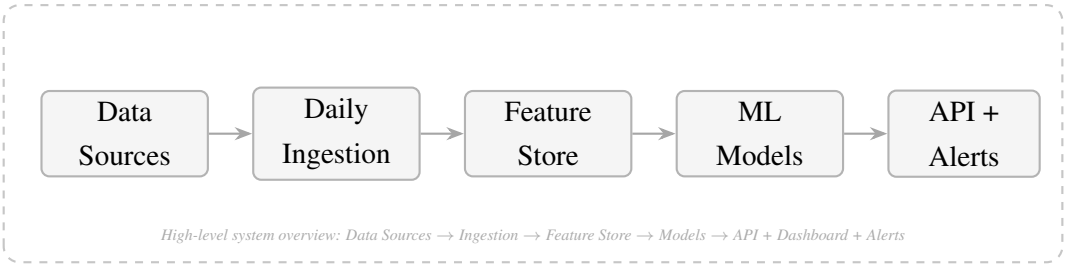


Figure 3.1: High-level overview of the ChiangMai Air Quality Alert System (CAAS).

## 3.2 Data Pipeline Design

### 3.2.1 Data Sources

The pipeline integrates three data sources, each providing a distinct signal relevant to PM2.5 forecasting in Northern Thailand.

Table 3.1: Summary of integrated data sources

| Source | Provider | Type | Coverage & Granularity |
|---|---|---|---|
| PM2.5 concentration | air4thai / PCD Thailand | Excel + REST API | 2011–2025, stations 35T & 36T, Chiang Mai (daily average) |
| Historical weather | Open-Meteo API | REST API | Chiang Mai coordinates (daily) |
| Fire hotspots | NASA FIRMS API | REST API | Northern Thailand bounding box (daily) |

The primary station is 35T (Chiang Mai City Hall, Chang Phueak), with station 36T

(Yupparaj School, Si Phum) serving as a fallback during gaps. The two stations exhibit a Pearson correlation of $r = 0.94$, confirming 36T as a reliable substitute for short-duration absences at the primary station. The full training dataset comprises approximately 5,475 daily observations spanning 15 years.

#### 3.2.2 Ingestion Method

Data ingestion follows a two-mode strategy. A one-time historical load parses all 15 PCD Excel files (one per year, 2011–2025) to build the initial training corpus. Ongoing ingestion runs daily at 06:00 local time via a scheduled GitHub Actions workflow, retrieving the previous day's data from all three sources before executing the forecast pipeline.

Batch ingestion (rather than streaming) is appropriate for this use case because daily forecast granularity provides no operational benefit from sub-hourly data updates.

#### 3.2.3 Transformation Pipeline

Transformations are applied in the following order within each pipeline execution:

1. **Schema validation** — verify expected columns, data types, and valid date range; pipeline halts immediately on any violation with no silent failures.
2. **Missing value handling** — interpolate PM2.5 gaps of up to three consecutive days using lag and rolling mean; substitute 36T station data for longer gaps; set fire count to zero on days with no detected hotspots.
3. **Unit normalisation** — standardise column names, numeric types, and date formats across all three source schemas.
4. **Feature engineering** — compute all derived features as described in Section 3.3.
5. **Target variable creation** — generate PM2.5 forward-shifted targets for $t + 1$, $t + 3$, and $t + 7$.

Orchestration is managed by GitHub Actions (cron: `0 6 * * *`). Each step logs success or failure explicitly, and checkpoints are saved after each step to enable pipeline resumption without re-running from scratch.

### 3.3 Feature Engineering

The full feature set comprises 44 engineered features organised into two groups: temporal autocorrelation features and contextual environmental features.

### 3.3.1 Temporal Features

Table 3.2: Temporal autocorrelation and calendar features

| Feature | Description |
|---|---|
| `PM25_lag1/2/3/7/14` | PM2.5 values 1–14 days prior; captures short-term autocorrelation |
| `PM25_roll3/7/14/30_mean` | Rolling mean over 3–30 day windows; captures medium-term trend |
| `PM25_roll7/30_std` | Rolling standard deviation; quantifies pollution volatility |
| `month_sin`, `month_cos` | Cyclical month encoding; avoids ordinality assumption |
| `day_sin`, `day_cos` | Cyclical day-of-year encoding |
| `is_haze_season` | Binary: 1 if January–April |
| `is_rainy_season` | Binary: 1 if May–October |
| `is_cool_season` | Binary: 1 if November–December |

### 3.3.2 Contextual Environmental Features

Table 3.3: Contextual meteorological and fire activity features

| Feature | Description |
|---|---|
| `north_avg_PM25` | Average PM2.5 across 15 Northern Thailand stations; regional smoke signal |
| `north_max_PM25` | Maximum PM2.5 across Northern Thailand stations |
| `wind_speed`, `wind_dir` | Wind speed and direction from Open-Meteo; governs smoke transport |
| `humidity`, `temperature` | Atmospheric conditions from Open-Meteo |
| `boundary_layer_height` | Atmospheric mixing height; key driver of surface PM2.5 accumulation |
| `fire_count` | Daily hotspot count within 100 km of Chiang Mai (NASA FIRMS) |
| `fire_frp_sum` | Total Fire Radiative Power; proxy for fire intensity and smoke volume |

### 3.3.3 Target Variables

Table 3.4: Target variables for regression and classification tasks

| Variable | Type | Definition |
|---|---|---|
| `PM25_next1` | Regression | PM2.5 concentration tomorrow (μg/m$^3$) |
| `PM25_next3` | Regression | PM2.5 concentration in 3 days |
| `PM25_next7` | Regression | PM2.5 concentration in 7 days |
| `alert_next1` | Binary classification | 1 if PM2.5 tomorrow $>$ 50 μg/m$^3$ |
| `alert_next3` | Binary classification | 1 if PM2.5 in 3 days $>$ 50 μg/m$^3$ |

## 3.4 Model Training and Evaluation

### 3.4.1 Model Selection Rationale

This project employs a two-model comparative strategy rather than assuming *a priori* that one architecture will outperform the other. This decision is motivated by an important consideration raised during proposal review: with approximately 5,475 daily

observations, does a sequential deep learning model offer genuine advantages over a well-tuned gradient boosting model with rich temporal features?

The honest answer is: not necessarily. The comparative experimental design allows the data to answer this question empirically.

*XGBoost — Primary Baseline Model*

XGBoost [4] is the primary baseline and a strong candidate for the production model. Its key advantages for this dataset include:

- Efficient and robust training on moderate-sized tabular data with limited overfitting risk.
- Native handling of missing values without imputation.
- Strong regularisation (L1/L2) to prevent overfitting on the approximately 5,475-sample training set.
- Interpretability via SHAP feature importance analysis, enabling model behaviour explanation to non-technical stakeholders.
- Given that the temporal features already encode autocorrelation and rolling patterns explicitly, XGBoost operates on a feature representation that has effectively "pre-digested" the sequence structure into a flat tabular form—which is precisely where gradient boosting excels.

*LSTM — Comparative Sequential Model*

An LSTM network [5] with a 30-day input window is deployed as a comparative sequential model. The rationale for including LSTM is not an expectation of across-the-board superiority, but a specific testable hypothesis:

**Hypothesis:** XGBoost is expected to perform strongly on t+1 forecasting, where the lag-1 feature is highly predictive and the fixed-window feature representation is adequate. LSTM may demonstrate advantages on longer-horizon forecasts (t+3, t+7), where multi-step sequential dependencies and the compounding of seasonal atmospheric signals may be better captured through explicit sequence modelling than through the static lag features available to XGBoost.

Regarding the sample size: while the dataset contains approximately 5,475 row-level observations, training an LSTM with a 30-day sliding window generates approximately 5,400 overlapping sequence samples of length 30. Regularisation measures including dropout (rate 0.2–0.3), L2 weight decay, and early stopping will be applied to mitigate overfitting risk. If LSTM training proves unstable or overfitting cannot be controlled, the XGBoost model will serve as the sole production model—a fully acceptable out-

come since the comparative evaluation is itself the contribution.

### 3.4.2 Chronological Data Split

A strict chronological split is used with no data shuffling, respecting the temporal dependency structure and simulating realistic deployment conditions.

Table 3.5: Chronological training, validation, and test split

| **Split** | **Period** | **Rationale** |
|---|---|---|
| Training | 2011–2022 | 12 years; includes multiple complete haze seasons |
| Validation | 2023 | Hyperparameter tuning; one complete annual cycle |
| Test | 2024–2025 | Fully unseen; includes the most recent haze seasons |

# CHAPTER 4

# SYSTEM ARCHITECTURE AND PIPELINE DESIGN

## 4.1 Architecture Overview

The CAAS architecture follows a layered design with clearly separated concerns across ingestion, storage, compute, serving, and monitoring layers. All components are deployed on AWS using the available Learner Lab credits.

Figure 4.1: AWS cloud architecture for the ChiangMai Air Quality Alert System (CAAS). The pipeline spans three external data sources (air4thai API, Open-Meteo API, NASA FIRMS API), an AWS Lambda ingestion layer triggered daily at 06:00 UTC+7 via GitHub Actions, four S3 storage tiers (Raw, Processed, Feature Store, Model Artifacts), and a compute layer comprising MLflow Tracking Server, FastAPI Inference Server, Streamlit Dashboard, and Evidently AI drift monitoring. Dashed arrows indicate orchestration triggers and automated retraining signals.

## 4.2 Storage Layers

The storage architecture separates data by processing stage into four layers on Amazon S3:

- **Raw Layer** (`s3://caas/raw/pm25/`, `raw/weather/`, `raw/fire/`) — Immutable, append-only source data exactly as ingested. Files are never overwritten.
- **Processed Layer** (`s3://caas/processed/merged/`) — Cleaned, validated,

and schema-enforced daily merged files.

- **Feature Store** (`s3://caas/features/pm25_daily_chiangmai.csv`) — ML-ready dataset with all 44 features and target columns, versioned by run date.
- **Model Artifacts** — MLflow artifact store containing trained model files, hyperparameters, evaluation metrics, SHAP plots, confusion matrices, and loss curves per experiment run.

## 4.3 MLOps Lifecycle

### 4.3.1 Experiment Tracking

All training runs are logged to MLflow, recording: hyperparameters (learning rate, `n_estimators`, LSTM units, sequence length, dropout rate), evaluation metrics (RMSE, MAE, $R^2$, Precision, Recall, F1, AUROC), artifacts (model file, feature importance plot, confusion matrix, loss curves), and the dataset version (snapshot date) used for training. This enables full reproducibility and direct side-by-side comparison between XGBoost and LSTM runs.

### 4.3.2 Model Versioning and Promotion

Models are managed through the MLflow Model Registry using a three-stage promotion workflow:

$$[\text{Staging}] \longrightarrow [\text{Validation}] \longrightarrow [\text{Production}]$$

- **Staging** — Newly trained model awaiting metric evaluation.
- **Validation** — Model has passed RMSE and Recall thresholds; ready for deployment review.
- **Production** — Currently serving model; only one model occupies this stage at any time.

Promotion to Production is automated when metrics pass predefined thresholds. Previous Production models are archived, not deleted, to enable rollback.

*MLflow Model Promotion Flow: Training → Staging → Validation Gate → Production / Archived*

Figure 4.2: MLflow model promotion and lifecycle flow. Newly trained models are registered in MLflow and enter the Staging stage, where they undergo schema validation (44-feature check), performance validation ($R^2 \geq 0.70$), and regression tests against the current Production model. Failed validation keeps the model in Staging with an alert logged. On success, the model is promoted to Production with zero-downtime FastAPI hot-reload; the previous Production model is archived for rollback. Evidently AI drift detection and a daily scheduled pipeline serve as the primary retraining triggers.

### 4.3.3 GitHub Actions CI/CD Retraining Pipeline

Proposal review raised the specific question of what stages exist in the GitHub Actions retraining workflow, which step blocks promotion, and whether there are charges. This subsection provides complete answers.

The retraining workflow (`.github/workflows/retrain.yml`) is triggered by three events: a cron schedule (December 1 annually), a webhook from the drift monitor when any threshold in Table 4.2 is exceeded, or a manual `workflow_dispatch` call. The pipeline comprises six sequential stages:

Table 4.1: GitHub Actions retraining pipeline stages

| # | Stage | Description and failure behaviour |
|---|---|---|
| 1 | `data-snapshot` | Pull latest PM2.5, weather, and FIRMS data; validate schema; save versioned snapshot to S3. *Failure: halt pipeline, send alert, no model update.* |
| 2 | `feature-build` | Run `build_features.py`; produce updated `features.csv`. *Failure: halt.* |
| 3 | `model-train` | Train XGBoost and LSTM candidate models on updated data. Log all runs to MLflow. *Failure: halt.* |
| 4 | **`validation-gate`** | **Blocking stage.** Compare candidate vs. champion on 90-day rolling window. Must satisfy: (a) MAE improvement $\geq 5\%$; (b) alert F1 $\geq 0.75$; (c) 45-feature schema match. *Failure: candidate stays in Staging; champion unchanged; failure reason logged.* |
| 5 | `mlflow-register` | Tag validated candidate as `Production`; archive previous champion as `Archived`. |
| 6 | `deploy` | Pull new `Production` model to EC2; zero-downtime FastAPI hot-reload via uvicorn `-reload`. |

**Regarding cost:** The project repository is public on GitHub. Public repositories receive unlimited GitHub Actions minutes at no charge. For a private repository, the free tier provides 2,000 minutes per month; a typical full retraining run (data pull + feature build + XGBoost + LSTM + validation) is estimated at 30–50 minutes. At a maximum of 4 triggered retraining runs per month during the haze season, this totals approximately 160–200 minutes—comfortably within the free tier, resulting in **$0 additional cost** for CI/CD.

### 4.3.4 Deployment Strategy

The deployment architecture uses a hybrid batch and real-time API approach. The primary daily pipeline runs at 06:00 via GitHub Actions: it ingests the previous day's data, generates t+1/3/7 forecasts, and stores results to S3. A FastAPI service exposes `/predict` and `/alert` endpoints for the Streamlit dashboard and external consumers.

*Daily Pipeline: Ingest → Validate → Clean → Features → Predict → Store → Monitor*

Figure 4.3: Daily MLOps pipeline execution flow triggered at 06:00 UTC+7 via GitHub Actions. The pipeline comprises seven stages: (1) Data Ingestion via AWS Lambda fetching from air4thai, Open-Meteo, and NASA FIRMS APIs into S3 Raw; (2) Schema Validation with a failure branch that sends an alert and stops the pipeline; (3) Data Cleaning including duplicate removal, forward-fill for missing values, and outlier clipping; (4) Feature Engineering producing 44 features (lag, rolling statistics, cyclical encodings, seasonal flags, fire hotspots) stored to S3 Feature Store; (5) Model Inference generating PM2.5 regression values and alert-level classifications for t+1, t+3, and t+7 horizons; (6) Store Predictions to S3 and update the FastAPI cache; (7) Monitoring Check using Evidently AI PSI drift detection, flagging for retraining if PSI $> 0.2$.

*Sequence Diagram: GitHub Actions trigger → Lambda → S3 → Feature build → Model inference → FastAPI*

Figure 4.4: End-to-end PM2.5 forecast request flow for the CAAS system. When a user opens the Streamlit Dashboard, it sends a `GET /forecast?station=ChiangMai&horizons=1,3,7` request to the FastAPI Inference Server. The server first queries the S3 Prediction Cache: on a cache hit, predictions for t+1, t+3, and t+7 days are returned immediately; on a cache miss, the server loads the Production model from MLflow, runs PM2.5 regression and alert-level classification inference, writes the results to the cache, and returns a JSON response. In parallel, Evidently AI performs background drift monitoring by comparing today's feature distribution against the training baseline using PSI scoring. If PSI exceeds 0.2, a drift alert triggers the GitHub Actions retraining pipeline via webhook.

## 4.4 Monitoring and Automated Retraining

This section provides a unified definition of drift detection and retraining logic, addressing feedback received during the proposal review that noted inconsistency between drift signals, thresholds, and retraining triggers.

### 4.4.1 Drift Detection Signals

Three independent drift signals are monitored, each targeting a distinct failure mode:

Table 4.2: Unified drift detection signals, thresholds, and actions

| Signal | Tool | Threshold | Action |
|---|---|---|---|
| 7-day rolling MAE on live predictions | MLflow + daily pipeline | >15 µg/m$^3$ (t+1) | Flag for retraining candidate; alert logged |
| Feature distribution drift (PSI) on `pm25_lag1`, `wind_speed`, `hotspot_50km` | Evidently AI (weekly) | PSI $> 0.2$ | Flag data drift; inspect source data quality |
| Kolmogorov-Smirnov test on 30-day residual distribution | Evidently AI (monthly) | $p < 0.05$ | Flag concept drift; trigger retraining |
| Scheduled (annual) | GitHub Actions (cron) | December 1 each year | Forced retrain incorporating latest full year |

### 4.4.2 Retraining Validation Gate

Retraining is triggered when *any* signal in Table 4.2 fires. However, a new model is not automatically promoted to production. The following gate must be passed before promotion:

1. **Schema validation**: candidate model accepts exactly the same 45-feature input schema.
2. **Performance gate**: candidate model must achieve MAE improvement of at least 5% over the current production champion on a 90-day rolling validation window.
3. **Alert recall gate**: candidate alert F1-score (threshold $>$ 50 µg/m$^3$) must be

$\geq 0.75$ on the validation window.

If the candidate fails any gate, it remains in MLflow `Staging` status and the champion continues to serve predictions. The pipeline logs the failure reason explicitly and raises a notification. This design ensures that automated retraining never silently degrades production accuracy.

### 4.4.3 Evidently AI Monitoring Configuration

Evidently AI runs as a **scheduled weekly report** (GitHub Actions cron), not as an always-on daemon. This eliminates the idle compute cost while still providing regular drift visibility. Reports are stored as HTML artifacts in S3 and linked from the Streamlit dashboard. Only when a report flags a threshold breach is any human or automated action triggered.

## 4.5 Infrastructure and Cost Estimate

Proposal review feedback noted inconsistency between the stated total cost and the individual component breakdown, and questioned whether always-on serving components (FastAPI, Streamlit, MLflow) are compatible with a low monthly budget. This section provides an accurate reconciliation.

**Key architectural decision**: FastAPI, Streamlit, and the MLflow Tracking Server all run on a *single shared* EC2 t3.micro instance. There is no separate instance per service. Evidently AI runs as a scheduled weekly batch job, not as an always-on process. This co-location is feasible because these services are lightweight and rarely active simultaneously under the expected load of one automated daily pipeline run and occasional human dashboard access.

Table 4.3: Revised monthly AWS infrastructure cost breakdown

| **Service** | **Usage detail** | **Monthly cost** |
|---|---|---|
| EC2 t3.micro (shared) | FastAPI + Streamlit + MLflow UI, always-on; ∼$0.0104/hr | $7.50 (on-demand) / $0 (Learner Lab) |
| Amazon S3 | ∼5 GB total (raw + processed + features + models + reports) | $0.12 |
| AWS Lambda | Ingestion trigger: 1 invocation/day × 5 min × 128 MB | $0.00 (well within free tier) |
| GitHub Actions | Orchestration: public repo = unlimited; private = ∼160 min/month (retraining) ≪ 2,000 free tier | $0.00 |
| Open-Meteo API | Free academic archive endpoint | $0.00 |
| NASA FIRMS API | Free with registered MAP key | $0.00 |
| **Total** | All services combined | **$7.62 – $8/month** |
| **(Learner Lab)** | EC2 covered by credits | **$0.12/month** |

The corrected total is **$8/month** under standard AWS pricing, or effectively **under $1/month** within the AWS Learner Lab environment. This replaces the previously stated $2–8 range, which was inconsistent with the EC2 cost alone exceeding the lower bound.

### 4.6 Data Quality Management

Data quality is treated as a first-class concern in the pipeline. Each ingested record must pass schema validation before entering the processing layer; any violation halts the pipeline immediately with an explicit error log and GitHub Actions email notification. Silent failures are not permitted.

### 4.6.1 Missing Value Handling Strategy

Table 4.4: Missing value handling by data source

| Source | Expected Missing Rate | Handling Strategy |
|---|---|---|
| air4thai PM2.5 (35T) | ∼2–3% per year | Gaps ≤3 consecutive days: linear interpolation using adjacent lag and rolling mean values. Gaps >3 days: substitute 36T station data ($r = 0.94$). All imputed rows flagged with `is_imputed = 1` column. |
| Open-Meteo weather | Near zero (reanalysis data) | No action required; reanalysis fills missing observations by design. |
| NASA FIRMS fire | Zero hotspots on some days is valid (not missing) | Fill `fire_count = 0` and `fire_frp_sum = 0` on days with no detected hotspots in the bounding box. |

### 4.6.2 Schema Validation Rules

Every pipeline run enforces the following validation checks before processing proceeds:

- Expected column names and data types are present and correct.
- Date range is within bounds and contains no duplicate dates.
- Numeric values fall within physically plausible ranges (e.g., PM2.5 $\in [0, 500]$ µg/m$^3$).
- Station identifiers match expected codes (35T, 36T).

Earlier PCD files (2011–2017) may differ in column count due to network expansion; the validator applies source-year-specific schemas to handle this gracefully.

### 4.6.3 Pipeline Health Monitoring

Each pipeline step (ingest → validate → preprocess → feature engineering → predict) emits structured logs containing: step name, timestamp, input row count, output row count, and pass/fail status. If any step fails, all downstream steps are blocked immediately. A data freshness check after each ingestion run verifies that the latest PM2.5 record date matches yesterday; if data is more than two days stale, the source is flagged as potentially unavailable and an alert is raised.

# CHAPTER 5

# EXPERIMENTAL PLAN AND EXPECTED CONTRIBUTIONS

## 5.1 Dataset Characteristics

The training dataset comprises approximately 5,475 daily observations from 2011 to 2025, representing 15 complete annual cycles. Key statistical characteristics include:

- **March 2025 average**: 55.1 μg/m$^3$ — more than $3\times$ the WHO 24-hour guideline.
- **Single-day maximum**: 92.3 μg/m$^3$ (March 23, 2025).
- **Days exceeding Thailand's 25 μg/m$^3$ threshold**: 109 days (31% of the year).
- **PM2.5 missing rate**: approximately 2–3% per year.
- **Cross-station correlation** (35T vs. 36T): $r = 0.940$.

## 5.2 Baseline Methods for Comparison

Three strategies will be compared:

1. **Persistence baseline** — the forecast equals the most recent observed PM2.5 value (lag-1). This naive baseline establishes the lower bound that any model must exceed.
2. **XGBoost with 44 engineered features** — the primary tabular baseline described in Chapter 3.
3. **LSTM with 30-day sequence window** — the comparative sequential model.

### 5.3 Evaluation Metrics

Table 5.1: Evaluation metrics by task type and priority

| **Task** | **Metrics** | **Priority and Rationale** |
|---|---|---|
| Regression (PM2.5 µg/m$^3$) | RMSE, MAE, $R^2$ | RMSE is the primary regression metric; $R^2 \geq 0.70$ is the target benchmark. |
| Classification (alert/no alert) | Precision, Recall, F1, AUROC | Recall is the primary classification metric. A missed true alert (false negative) carries greater public health cost than a false alarm (false positive). |

### 5.4 Experimental Setup

Experiments will be conducted as follows:

1. Both models trained on the 2011–2022 training set; hyperparameters tuned on the 2023 validation set using random search.
2. Final evaluation on the 2024–2025 test set (fully unseen data).
3. Separate experiments for each forecast horizon (t+1, t+3, t+7) to test whether any LSTM advantage is horizon-dependent.
4. XGBoost SHAP analysis conducted to identify the most influential features for interpretability and domain insight.
5. All runs logged to MLflow with full hyperparameter, metric, and artifact records for reproducibility.

### 5.5 Expected Contributions

Following proposal review feedback, this section accurately frames the project's contribution. CAAS is not primarily a novel forecasting methodology—the models employed (XGBoost, LSTM) are established techniques. The contribution lies in the **production-grade integration** of multiple components into a coherent, automated, and

self-maintaining operational system. This is precisely what the MLOps course is designed to develop and evaluate.

1. **A production-ready MLOps reference pipeline** for environmental forecasting, demonstrating the complete operational lifecycle: multi-source ingestion → feature engineering → chronologically-split model training → MLflow experiment tracking → staged model promotion with validation gates → FastAPI serving → Streamlit public dashboard → automated drift detection and retraining. This end-to-end integration, applied to a genuine public-health problem, is the primary contribution.

2. **An evidence-based model comparison** of XGBoost (tabular gradient boosting) versus LSTM (sequential deep learning) on 15 years of daily PM2.5 data, evaluated separately per forecast horizon (t+1, t+3, t+7). The production champion is selected empirically, not assumed in advance.

3. **A reproducible 45-feature engineering framework** integrating PM2.5 autocorrelation, rolling statistics, cyclical calendar encodings, meteorological variables, and NASA FIRMS satellite fire hotspot signals into a unified ML-ready dataset.

4. **A public health early-warning tool** providing Chiang Mai residents with 1-, 3-, and 7-day PM2.5 forecasts and automated hazardous-level alerts via a Streamlit dashboard.

### 5.6 Feasibility and Tool Selection

All tools are open-source, well-documented, and either free or within Learner Lab credit limits. The technology stack is:

- **Language**: Python 3.11
- **ML models**: XGBoost, TensorFlow/Keras (LSTM)
- **Experiment tracking**: MLflow
- **API serving**: FastAPI
- **Dashboard**: Streamlit
- **Drift monitoring**: Evidently AI
- **Orchestration / CI-CD**: GitHub Actions
- **Cloud**: AWS S3, Lambda, EC2 t3.micro (Learner Lab)
- **Data sources**: air4thai API (PCD), Open-Meteo API, NASA FIRMS API

The primary feasibility risk is LSTM training stability and overfitting given the dataset size. This risk is mitigated by the parallel XGBoost strategy: if LSTM proves problematic, XGBoost provides a fully capable production model.

### 5.7 Team Roles and Responsibilities

Table 5.2: Team member roles and task ownership

| Member | Student ID | Responsibilities |
|---|---|---|
| Supanut Kompayak | st126055 | Data pipeline design and implementation; feature engineering; model training (XGBoost + LSTM); experiment tracking (MLflow). |
| Shuvam Shrestha | st125975 | FastAPI endpoint development; Streamlit dashboard; AWS cloud architecture design; cost estimation; CI/CD pipeline (GitHub Actions); Evidently AI monitoring setup; infrastructure deployment and maintenance. |

### 5.8 Known Limitations and Risk Mitigation

Transparency about the project's limitations is essential for realistic feasibility assessment. Table 5.3 summarises the known risks and planned mitigations.

Table 5.3: Known limitations and risk mitigation strategies

| **Risk** | **Description** | **Mitigation** |
|---|---|---|
| Dataset size for LSTM | 5,475 daily observations is in the moderate-size regime where deep learning models are known to sometimes underperform well-tuned gradient boosting methods. | XGBoost serves as a fully capable fallback production model. Dropout and early stopping regularise LSTM. Results are reported honestly regardless of outcome. |
| air4thai API access | The air4thai API (`air4thai.com`) is blocked outside Thai IP ranges, limiting automated ingestion from cloud infrastructure hosted outside Thailand. | Historical data (2011–2025) acquired via direct Excel file download. For ongoing ingestion, a lightweight script runs on a Thai IP (e.g., AIT campus) or via VPN, then pushes to S3. |
| AWS Learner Lab expiry | Learner Lab credits may expire or reset during the semester, potentially disrupting always-on services. | All infrastructure is defined as code (GitHub Actions, configuration files) enabling rapid redeployment. MLflow and model artifacts are persisted to S3 to survive instance restarts. |
| Concept drift | Long-term shifts in haze season onset, duration, and intensity due to climate change may cause model performance to degrade between training and future operational use. | Annual scheduled retraining in December each year incorporates the most recent full year of data to reflect current environmental patterns. |
| Single-station dependency | If both 35T and 36T stations simultaneously experience extended outages, the pipeline has no PM2.5 source. | The data freshness check will raise an alert within 48 hours. Manual data acquisition from alternative PCD stations can be used as an emergency fallback. |

### 5.9 Revised Project Timeline

The timeline below reflects actual progress as of March 27, 2026 (progress report submission date) and realistic targets for the remaining 7 days.

Table 5.4: Updated project milestones with completion status

| Milestone | Owner | Target Date | Status |
|---|---|---|---|
| Proposal submission | Both | Mar 12 | Done |
| PM2.5 data parsed (5,478 rows) | Supanut Kompayak | Mar 15 | Done |
| NASA FIRMS data fetched | Supanut Kompayak | Mar 27 | Done |
| Weather data fetched (Open-Meteo) | Supanut Kompayak | Mar 28 | In progress |
| Feature engineering script (45 features) | Supanut Kompayak | Mar 27 | Done |
| Progress report submitted | Both | Mar 27 | Done |
| XGBoost baseline trained + MLflow logged | Supanut Kompayak | Mar 29 | Upcoming |
| LSTM model trained + compared | Supanut Kompayak | Apr 1 | Upcoming |
| FastAPI inference endpoint | Shuvam Shrestha | Apr 1 | Upcoming |
| Streamlit dashboard deployed | Shuvam Shrestha | Apr 1 | Upcoming |
| AWS infrastructure + CI/CD live | Shuvam Shrestha | Apr 3 | Upcoming |
| Drift monitoring live (Evidently) | Shuvam Shrestha | Apr 3 | Upcoming |
| Final report + presentation | Both | TBD | Upcoming |

# CHAPTER 6

# IMPLEMENTATION PROGRESS

This chapter describes the concrete implementation work completed as of the progress report submission date (March 27, 2026).

## 6.1 Data Collection Status

Table 6.1: Data collection and processing status as of March 27, 2026

| Dataset | Source | Status | Notes |
|---|---|---|---|
| PM2.5 historical (2011–2025) | PCD Excel | **Complete** | 5,478 rows; 4.0% missing; stations 35T/36T merged |
| Weather variables (2011–2025) | Open-Meteo API | In progress | Script complete; awaiting execution on unrestricted network |
| Fire hotspots (2011–2025) | NASA FIRMS API | **Complete** | Aggregated to daily; 50 km and 100 km radius counts |
| Merged feature matrix | Derived | **Complete (partial)** | 45 features; weather columns temporarily NaN; 5,256 usable rows |

### 6.1.1 PM2.5 Data: Station Merging Logic

The primary monitoring station is 35T (Chiang Mai City Hall, Chang Phueak). Station 35T data is unavailable for 2011–2015 (missing rate: 64% in 2011, full absence in 2012–2015). Station 36T (Yupparaj School, Si Phum) serves as the primary source for those years. The two stations exhibit $r = 0.940$ Pearson correlation where concurrent data is available, confirming 36T as a reliable substitute. The implemented logic is:

- **2011–2016**: use 36T as primary (35T unavailable or unreliable).
- **2017–2025**: use 35T as primary; fall back to 36T for individual missing days.
- **All years**: forward-fill gaps of $\leq 3$ consecutive days; longer gaps use 36T substitution; all imputed rows flagged with `station_source` column.

### 6.1.2 NASA FIRMS Integration

Fire hotspot data was obtained from the FIRMS MODIS Standard Product (MODIS_SP) using the bounding box covering Northern Thailand and cross-border fire regions (97.0°E–101.0°E, 17.0°N–21.5°N). This extended bounding box is necessary because trans-boundary fire smoke from Myanmar and Laos is a documented major contributor to Chiang Mai PM2.5 episodes. Daily aggregation computes:

- `hotspot_50km`: count of fire pixels within 50 km of Station 35T (Haversine distance).
- `hotspot_100km`: count within 100 km radius.
- `mean_frp_50km`: mean Fire Radiative Power (MW) within 50 km—a proxy for fire intensity.
- `hotspot_7d/14d_roll`: 7- and 14-day rolling mean of `hotspot_50km`.

Days with no detected hotspots are recorded as zero (not missing), correctly representing fire-free days.

## 6.2 Scripts Inventory

Table 6.2: Completed scripts in `04_Scripts/`

| Script | Purpose | Status |
|---|---|---|
| `parse_pm25.py` | Parse 15 PCD Excel files; merge 35T/36T; output CSV | Complete |
| `fetch_weather.py` | Fetch 11 weather vars from Open-Meteo archive API | Complete |
| `fetch_firms.py` | Fetch MODIS hotspot data; aggregate by radius | Complete |
| `build_features.py` | Merge all sources; engineer 45 features + 6 targets | Complete |
| `train_xgboost.py` | Train XGBoost per horizon; log to MLflow | Complete |
| `train_lstm.py` | Train LSTM per horizon; log to MLflow | Complete |

## 6.3 Project Folder Structure

```
Proposal Presentation/
```

```
|-- .env                    (API keys - not committed to git)
|-- .env.example            (template)
|-- .gitignore
|-- requirements.txt        (full Python dependency list)
|-- 01_Proposal/            (proposal + progress report LaTeX
   )
|-- 02_Presentation/        (slide deck)
|-- 03_Data/
|   |-- raw/pm25/           (15 PCD Excel files, 2011-2025)
|   +-- processed/
|       |-- pm25_consolidated.csv    (5,478 rows)
|       |-- firms_consolidated.csv   (5,478 rows)
|       |-- weather_chiang_mai.csv   (pending fetch)
|       +-- features.csv             (5,256 rows, 45
   features)
|-- 04_Scripts/             (6 scripts as above)
|-- 05_Reference/           (rubric, guidelines, prior work)
|-- mlruns/                 (MLflow tracking store)
+-- FINALSUBMIT/            (submitted proposal PDF + slides)
```

# CHAPTER 7

# PRELIMINARY RESULTS

This chapter presents preliminary results from exploratory data analysis and persistence baseline evaluation. These results establish the performance floor that the XGBoost and LSTM models must exceed and provide insight into the structure of the PM2.5 time series that motivates the feature engineering design.

## 7.1 PM2.5 Dataset Characteristics

### 7.1.1 Overall Statistics

Table 7.1 summarises the consolidated 15-year PM2.5 dataset.

Table 7.1: Descriptive statistics for PM2.5 dataset (2011–2025, $n = 5{,}257$ valid days)

| **Statistic** | **Value** |
|---|---:|
| Total rows (daily observations) | 5,478 |
| Valid PM2.5 readings | 5,257 |
| Missing values | 221 (4.0%) |
| Mean PM2.5 | 29.9 μg/m$^3$ |
| Median PM2.5 | 21.0 μg/m$^3$ |
| Standard deviation | 25.6 μg/m$^3$ |
| Minimum | 3.0 μg/m$^3$ |
| Maximum | 266.0 μg/m$^3$ (March 16, 2015) |
| 25th percentile | 14.0 μg/m$^3$ |
| 75th percentile | 35.0 μg/m$^3$ |
| Days exceeding WHO guideline (>15 μg/m$^3$) | 3,630 (69.1%) |
| Days exceeding Thai standard (>25 μg/m$^3$) | 2,044 (38.9%) |
| Days in hazardous range (>50 μg/m$^3$) | 770 (14.6%) |
| Days in very hazardous range (>75 μg/m$^3$) | 317 (6.0%) |

The high standard deviation (25.6) relative to the median (21.0) indicates a right-skewed distribution driven by acute haze events. The mean (29.9) is pulled upward by these

extreme events, confirming that most days are below the Thai standard but that severe episodes are common enough to represent a meaningful public health burden.

### 7.1.2 Seasonal Pattern

PM2.5 in Chiang Mai exhibits a strongly seasonal pattern. Table 7.2 shows the average PM2.5 by calendar month across all 15 years.

Table 7.2: Average daily PM2.5 by month (2011–2025 mean)

| **Month** | **Mean PM2.5 (μg/m$^3$)** | **Season** | **Key driver** |
|---|---:|---|---|
| January | 33.2 | Haze | Early fire season; dry conditions |
| February | 46.8 | Haze | Increasing agricultural burning |
| March | 78.0 | Haze (peak) | Peak fire activity; maximum smoke accumula |
| April | 59.5 | Haze | Declining fires; early rains begin |
| May | 25.5 | Transition | Rains begin; pollution disperses |
| June | 14.9 | Wet | Monsoon; rain scavenging reduces PM2.5 |
| July | 14.3 | Wet | Peak monsoon |
| August | 14.4 | Wet | Continued monsoon |
| September | 13.7 | Wet | Late monsoon |
| October | 18.3 | Transition | Drying; regional fires begin |
| November | 20.9 | Cool | Crop residue burning commences |
| December | 28.4 | Cool/early haze | Increasing biomass burning |

March is the most critical month with a 15-year mean of 78.0 μg/m$^3$, more than five times the WHO 24-hour guideline. During the haze season (January–April), the mean is 54.3 μg/m$^3$ with 731 days exceeding the 50 μg/m$^3$ hazardous threshold, compared to only 39 such days in the remaining eight months. This bimodal annual structure strongly motivates the `is_haze_season` binary feature and cyclical seasonal encodings in the feature set.

### 7.1.3 Annual Trends

Table 7.3: Annual PM2.5 statistics (2011–2025)

| Year | Annual mean (µg/m$^3$) | Annual max | Missing days |
|---|---|---|---|
| 2011 | 14.9 | 33.0 | 143 |
| 2012 | 28.6 | 147.0 | 12 |
| 2013 | 34.6 | 188.0 | 18 |
| 2014 | 33.1 | 188.0 | 23 |
| 2015 | 32.7 | 266.0 | 12 |
| 2016 | 32.0 | 144.0 | 9 |
| 2017 | 29.0 | 114.0 | 3 |
| 2018 | 29.3 | 105.0 | 0 |
| 2019 | 36.2 | 228.0 | 0 |
| 2020 | 33.9 | 174.0 | 1 |
| 2021 | 29.0 | 131.0 | 0 |
| 2022 | 23.2 | 72.0 | 0 |
| 2023 | 34.4 | 220.0 | 0 |
| 2024 | 30.1 | 156.5 | 0 |
| 2025 | 22.8 | 92.3 | 0 |

The dataset shows year-to-year variability driven by fire activity, with extreme years including 2015 (max 266.0 µg/m$^3$), 2019 (max 228.0), and 2023 (max 220.0). The high missing rate in 2011 (143 days) reflects the limited availability of Station 36T data in the earliest monitoring period; this is fully handled by the station merging logic described in Chapter 6. The test set years (2024–2025) represent a diverse sample with one moderate (2025: mean 22.8) and one higher (2024: mean 30.1) pollution year.

## 7.2 Temporal Autocorrelation Analysis

The degree to which past PM2.5 values predict future values directly determines the theoretical effectiveness of lag-based features. Table 7.4 shows the autocorrelation coefficients at key lags.

Table 7.4: PM2.5 autocorrelation at selected lags

| Lag (days) | Autocorrelation ($r$) | Implication |
|---|---|---|
| 1 | 0.925 | Very strong; yesterday's PM2.5 is highly predictive for tomorrow |
| 3 | 0.791 | Strong; meaningful signal for t+3 forecasting |
| 7 | 0.723 | Moderate-strong; still useful for weekly forecasting |
| 14 | 0.631 | Moderate; haze episodes persist across 2-week periods |
| 30 | 0.456 | Weak; captures broad seasonal signal |

The exceptionally high lag-1 autocorrelation ($r = 0.925$) establishes that the lag-1 feature will be the single most important predictor in both models. This also explains why the persistence baseline ("forecast = yesterday's value") performs surprisingly well at t+1. The declining autocorrelation at longer lags motivates the inclusion of meteorological and fire hotspot features: when persistence weakens, contextual signals provide the margin needed for accurate t+3 and t+7 forecasting.

### 7.3 Persistence Baseline Evaluation

The persistence (naïve lag) baseline establishes the performance floor that all ML models must exceed to justify their additional complexity. Tables 7.5 and 7.6 show baseline performance overall and during the critical haze season.

Table 7.5: Persistence baseline performance across full dataset (2011–2025)

| Forecast horizon | Baseline | MAE (µg/m$^3$) | RMSE | $R^2$ |
|---|---|---|---|---|
| t+1 | lag-1 (yesterday's PM2.5) | 5.30 | 10.03 | 0.847 |
| t+3 | lag-3 (3 days ago) | 8.97 | 16.58 | 0.582 |
| t+7 | lag-7 (7 days ago) | 10.93 | 19.14 | 0.443 |

Table 7.6: Persistence baseline performance during haze season (January–April only)

| Horizon | Baseline | MAE (µg/m$^3$) |
|---|---|---|
| t+1 | lag-1 | 10.42 |
| t+3 | lag-3 | 18.51 |
| t+7 | lag-7 | 22.66 |

Table 7.7: Alert classification: persistence baseline for hazard alert (>50 µg/m$^3$ threshold)

| **Baseline** | **Precision** | **Recall** | **F1** |
|---|---|---|---|
| Persistence lag-1 (alert if yesterday > 50) | 0.844 | 0.844 | 0.844 |

**Interpretation:** The persistence baseline achieves $R^2 = 0.847$ at t+1, reflecting the high autocorrelation. However, during the haze season—the period most critical for public health—MAE degrades to 10.42 µg/m$^3$ at t+1 and 22.66 µg/m$^3$ at t+7. Given that alert thresholds are at 50 µg/m$^3$, a 22.66 µg/m$^3$ error at t+7 is clinically significant. These numbers define the target improvement that XGBoost and LSTM models must achieve. The alert baseline F1 of 0.844 provides the benchmark for the classification task.

## 7.4 Target Performance Benchmarks

Based on the persistence baselines and the autocorrelation structure, the following quantitative targets are set for the ML models:

Table 7.8: Minimum performance targets for XGBoost and LSTM models

| **Horizon** | **Task** | **Baseline MAE** | **Target MAE** | **Target $R^2$** | **Target F1** |
|---|---|---|---|---|---|
| t+1 | Regression | 5.30 | <4.5 | >0.88 | — |
| t+1 | Alert cls. | — | — | — | >0.88 |
| t+3 | Regression | 8.97 | <7.5 | >0.70 | — |
| t+3 | Alert cls. | — | — | — | >0.78 |
| t+7 | Regression | 10.93 | <9.5 | >0.58 | — |
| t+7 | Alert cls. | — | — | — | >0.75 |

All targets represent a minimum 10–15% improvement over the persistence baseline. These benchmarks directly inform the validation gate in the CI/CD retraining pipeline (Table 4.1).

# CHAPTER 8

# DETAILED EXPERIMENTAL SCENARIOS

This chapter addresses proposal review feedback requesting that scenarios and experiments be described in detail.

## 8.1 Overview of Experimental Design

The experimental design comprises four scenarios. Each scenario tests a distinct research question and produces evidence that informs production model selection.

Table 8.1: Summary of experimental scenarios

| # | Scenario | Research question |
|---|---|---|
| A | XGBoost vs. Persistence | Does a well-tuned gradient boosting model meaningfully improve on the naïve baseline? |
| B | XGBoost vs. LSTM | Does sequential modelling offer advantages over tabular gradient boosting, and does any advantage vary by horizon? |
| C | Alert Classification | Which model more reliably detects hazardous episodes? What is the precision-recall trade-off? |
| D | Feature Ablation (FIRMS signal) | Does including NASA FIRMS satellite fire hotspot data improve forecast accuracy, particularly during haze season? |

All scenarios share the same chronological split: training on 2011–2022, validation on 2023, and test evaluation on 2024–2025. No data leakage across splits is possible because all features are computed using only past observations (shift/lag operations with positive lag values).

## 8.2 Scenario A: XGBoost versus Persistence Baseline

**Purpose:** Establish whether the proposed model pipeline delivers meaningful improvement over the naïve forecast.

**Setup:**

- Model: XGBoost with 45-feature input, hyperparameters tuned on 2023 validation set via random search (50 iterations).
- Baseline: Persistence (lag-1, lag-3, lag-7 as appropriate for each horizon).

- Evaluation set: 2024–2025 test split (731 days).
- Primary metrics: MAE, RMSE, $R^2$.
- Secondary metric: SHAP feature importance to identify which features drive improvement over persistence.

**Expected outcome:** XGBoost is expected to outperform persistence at all horizons, with the largest relative improvement at t+3 and t+7 where persistence degrades most sharply. If XGBoost cannot outperform persistence at t+7 by at least 10% MAE, that result is reported as a genuine finding with analysis.

### 8.3 Scenario B: XGBoost versus LSTM by Horizon

**Purpose:** Empirically determine whether explicit sequence modelling (LSTM) provides advantages over tabular gradient boosting, and whether any advantage is horizon-dependent.

**Hypothesis:**

- At t+1, XGBoost is expected to perform at least as well as LSTM. The lag-1 feature ($r = 0.925$) dominates, and XGBoost already "knows" the previous 30 days via lag and rolling features.
- At t+3 and t+7, LSTM may demonstrate advantages by capturing multi-step sequential dependencies and non-linear temporal interactions that fixed-lag features cannot represent. This hypothesis will be evaluated empirically.

**Setup:**

- XGBoost: 45 tabular features, median imputation for missing values, no temporal ordering in training.
- LSTM: 30-day input sequence ($\times$ 45 features), two LSTM layers (64 and 32 units), dropout 0.2, Huber loss, Adam optimiser, early stopping (patience $= 15$), all scaled via `StandardScaler` fit on training set only.
- Separate model fitted per horizon (t+1, t+3, t+7) for both architectures (six models total).
- Hyperparameters tuned on 2023 validation set.
- All runs logged to MLflow with full hyperparameter and metric records.

**Decision rule:** The model with lower test MAE on the 2024–2025 holdout is selected as the production champion for that horizon. If models tie within 5% MAE, XGBoost is preferred for interpretability.

### 8.4 Scenario C: Alert Classification

**Purpose:** Evaluate both models' ability to classify days as "hazardous" (PM2.5 $>$ 50 µg/m$^3$) versus "safe".

**Threshold rationale:** 50 µg/m$^3$ is Thailand's national "unhealthy" threshold and the point at which health authorities recommend the general population reduce outdoor activity. This is the most actionable threshold for public health communication.

**Setup:** Alert predictions are derived from regression outputs by applying a $>$ 50 µg/m$^3$ threshold to the point forecast. No separate classification head is trained; alert labels follow directly from regression predictions. This approach ensures the regression and alert outputs are internally consistent.

**Metric priority:** Recall is the primary classification metric. A missed true alert (false negative) carries significantly greater public health cost than a false alarm (false positive): false negatives prevent residents from taking protective action before a hazardous event; false positives cause unnecessary inconvenience but no direct health harm. The target recall is $\geq 0.85$ across all horizons.

Table 8.2: Alert classification evaluation targets per horizon

| Horizon | Target Recall | Target Precision | Target F1 | Target AUROC |
|---|---|---|---|---|
| t+1 | ≥0.85 | ≥0.80 | ≥0.82 | ≥0.92 |
| t+3 | ≥0.80 | ≥0.75 | ≥0.77 | ≥0.88 |
| t+7 | ≥0.75 | ≥0.70 | ≥0.72 | ≥0.85 |

### 8.5 Scenario D: Feature Ablation — FIRMS Fire Hotspot Signal

**Purpose:** Quantify the marginal contribution of NASA FIRMS satellite fire hotspot features to forecast accuracy. This tests whether the effort of integrating a third external data source is empirically justified.

**Setup:**

- Model A (*baseline*): XGBoost trained on PM2.5 lag/rolling + temporal + weather features only (39 features; all FIRMS features excluded).
- Model B (*with FIRMS*): XGBoost trained on the full 45-feature set including `hotspot_50km`, `hotspot_100km`, `mean_frp_50km`, rolling hotspot averages, and `fire_flag`.
- Both models evaluated on the same 2024–2025 test split.

- Results compared overall and specifically for haze-season days (January–April), where the fire signal is expected to provide the greatest marginal benefit.

**Hypothesis:** The FIRMS signal is expected to improve accuracy most during active fire periods (February–April), when fire activity is a primary driver of PM2.5 spikes that cannot be fully predicted from meteorological and autoregressive features alone. A SHAP analysis on Model B will confirm whether fire features rank among the top-10 most important during the haze season.

### 8.6 MLflow Experiment Organisation

All four scenarios are tracked in MLflow under the following experiment structure:

- Experiment `CAAS-XGBoost`: all XGBoost runs (Scenarios A, B, D); runs tagged by `horizon` and `scenario`.
- Experiment `CAAS-LSTM`: all LSTM runs (Scenario B); runs tagged by `horizon`.
- Model Registry: champion models promoted to `Production`; all others remain in `Staging` or `Archived`.

Each run logs: hyperparameters (including `n_features`, `train_end`, `val_end`), all metrics (MAE, RMSE, $R^2$, Precision, Recall, F1, AUROC per horizon), feature importance CSV, and prediction CSV for the test split. This ensures full reproducibility and direct numerical comparison between all scenario results.

# REFERENCES


[1] World Health Organization (WHO). (2021). *WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. World Health Organization. Geneva.

[2] Seinfeld, J. H., & Pandis, S. N. (2016). *Atmospheric Chemistry and Physics: From Air Pollution to Climate Change* (3rd ed.). John Wiley & Sons, New Jersey.

[3] Liang, Y., Ke, S., Zhang, J., Yi, X., & Zheng, Y. (2018). GeoMAN: Multi-level attention networks for geo-sensory time series prediction. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI-18)*, pp. 3428–3434. AAAI Press.

[4] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. ACM, New York.

[5] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

[6] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). The M4 competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4), 802–808.

[7] Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90.

[8] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 28. Curran Associates.

[9] Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Murching, A., Nykodym, T., Ogilvie, P., Parkhe, M., Singh, A., Xie, C., & Zumar, C. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39–45.

[10] Pollution Control Department, Thailand. (2024). *Air Quality and Noise Management: Air4Thai Monitoring System*. Retrieved from http://air4thai.com

[11] NASA EOSDIS. (2024). *Fire Information for Resource Management System (FIRMS)*. NASA Earthdata. Retrieved from https://firms.modaps.eosdis.nasa.gov

[12] Zippenfenig, P. (2023). Open-Meteo.com Weather API. *Zenodo*. https://doi.org/10.5281/zenodo.7970649